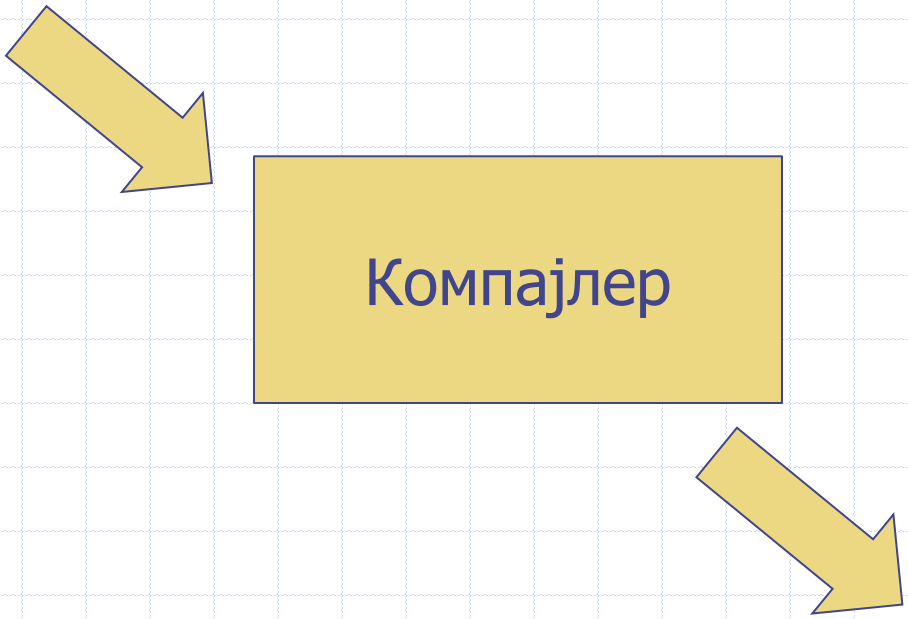
ПУЊАЧ ПРОГРАМА У МЕМОРИЈУ

- Основни задаци пуњача
- Типови пуњача

Улазни програм

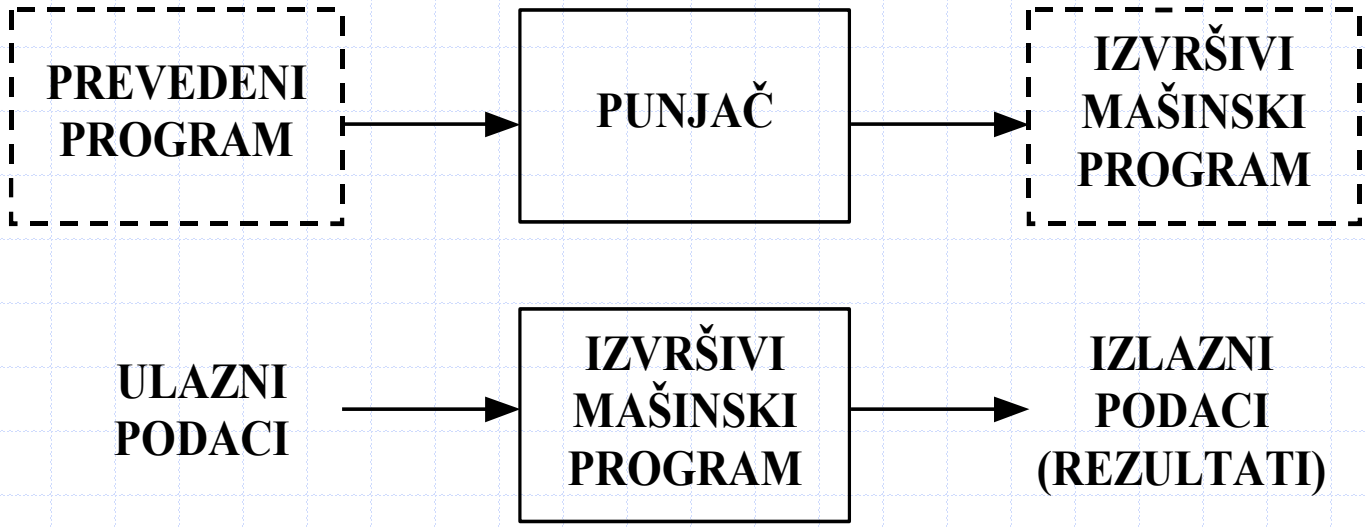


Објектна
датотека
(машински код)

Зашто је пуњач неопходан

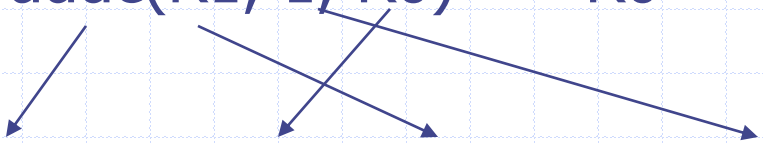
- Сви преведени програми имају почетну адресу нула, што је у мултипрограмским условима немогуће реализовати.
- Исто тако, програми могу да садрже један или више потпрограма.
- Зато се дефинише посебна компонента програмске подршке чији је задатак да пуни програме на различите адресе у оперативној меморији.

ПУЊАЧ: Програм који смешта друге програме у меморију.



Пример инструкције

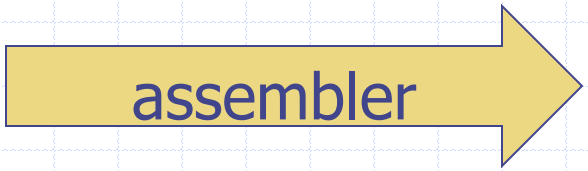
addc(R1, 1, R0) R0 <- R1 + 1



```
110000 00000 00001 0000000000000001
0xC001 0001
```

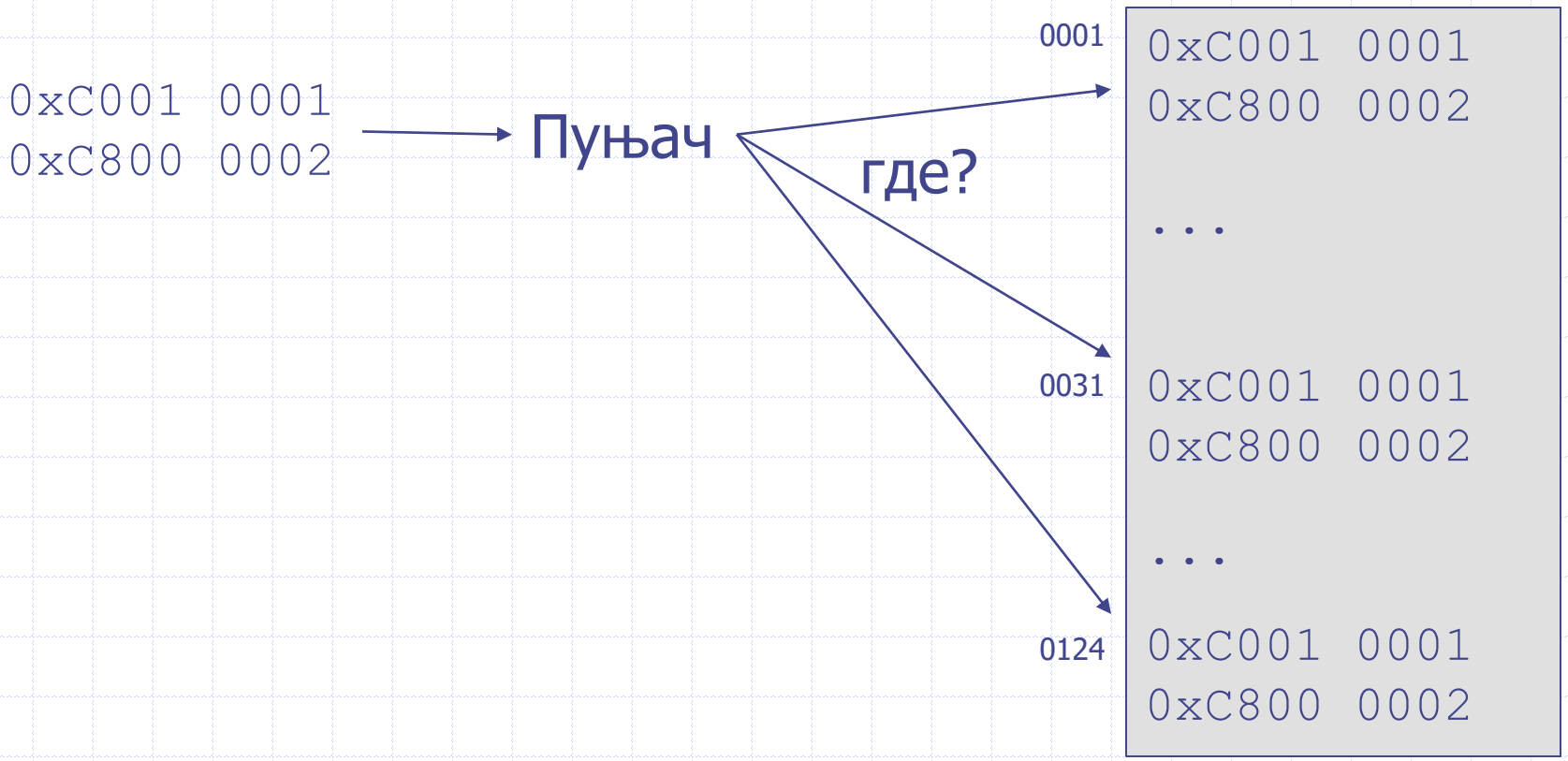
Пример инструкције

addc(R1, 1, R0)
mulc(R0, 2, R0)



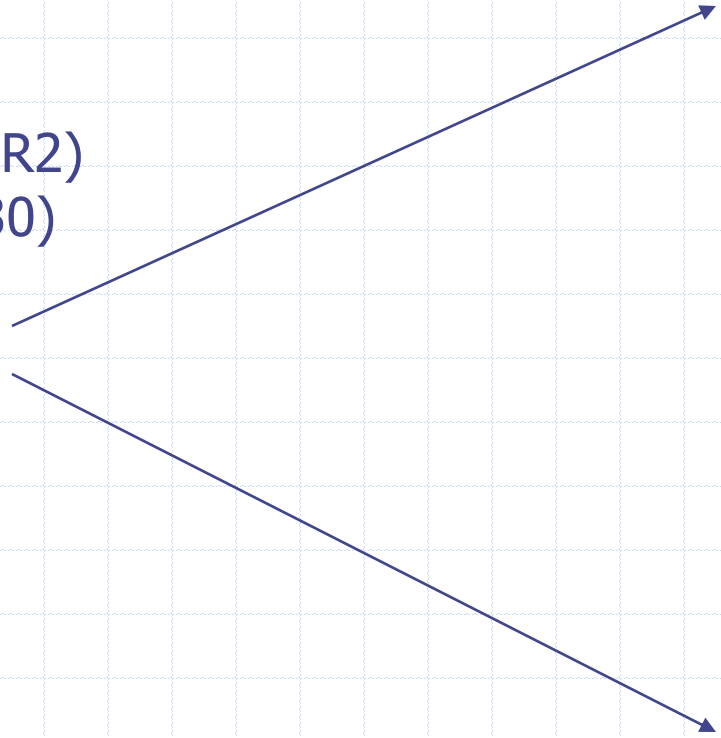
0xC001 0001
0xC800 0002

Смештање у оперативну меморију



Смештање у оперативну меморију

```
  cmpeqc(R1, 100, R2)
  beq(R2, Label, R30)
  addc(R1, 1, R0)
  mulc(R0, 2, R0)
Label:
  addc(R1, 3, R0)
  mulc(R0, 4, R0)
  ...
```



0001	0xd041	0064
0002	0x73c2	**0005**
0003	0xc001	0001
0004	0xc800	0002
0005	0xc001	0003
0006	0xc800	0004
0034	0xd041	0064
0035	0x73c2	**0038**
0036	0xc001	0001
0037	0xc800	0002
0038	0xc001	0003
0039	0xc800	0004

Релокација

- РЕЛОКАЦИЈА програма: размештај програма, почев од прве слободне адресе.
- Пуњач релоцира програм додавањем константе на сваку релокатибилну адресу у програму.
- Програм на излазу преводиоца је релокатибилан програм, а извршиви машински програм је апсолутни.

Сегмент

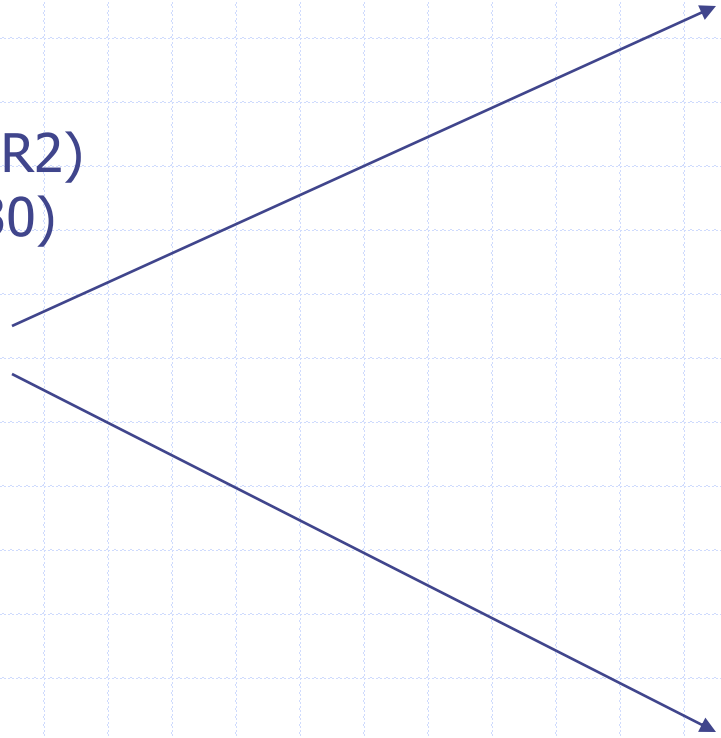
- **Сегмент** је скуп речи поређаних једна за другом.
- Сегмент је **најмања јединица** за смештање програма или података коју преводилац и пуњач препознају.

Повезивање

- Унутар главног програма могу постојати позиви/скокови на различите потпрограме или програме преведене у различито време и писане на различитим програмским језицима.
- Повезивање свих програма (потпрограма) у јединствен програм обухвата одређивање: (1) адреса за екстерне референце и (2) екстерних вредности.

Смештање у оперативну меморију

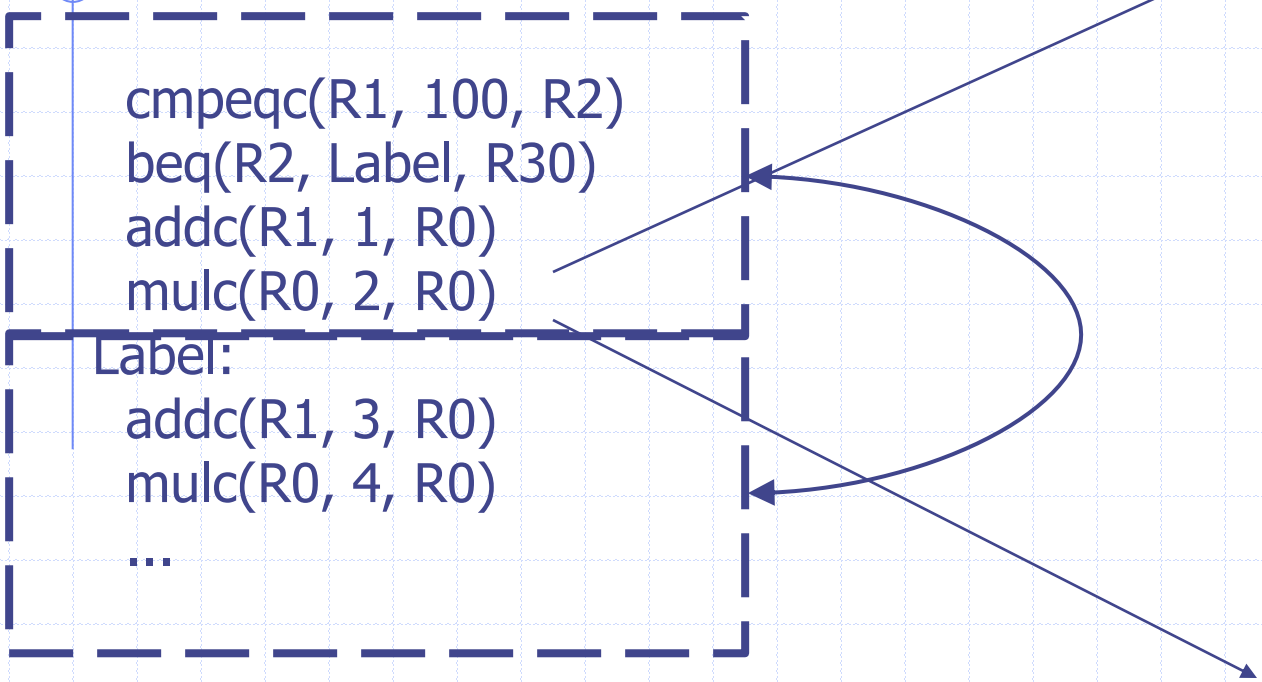
```
  cmpeqc(R1, 100, R2)
  beq(R2, Label, R30)
  addc(R1, 1, R0)
  mulc(R0, 2, R0)
Label:
  addc(R1, 3, R0)
  mulc(R0, 4, R0)
  ...
```



0001	0xd041	0064
0002	0x73c2	**0003**
0003	0xc001	0001
0004	0xc800	0002
0005	0xc001	0003
0006	0xc800	0004
0034	0xd041	0064
0035	0x73c2	**0003**
0036	0xc001	0001
0037	0xc800	0002
0038	0xc001	0003
0039	0xc800	0004

Ако је beq релативан скок.

Смештање у оперативну меморију

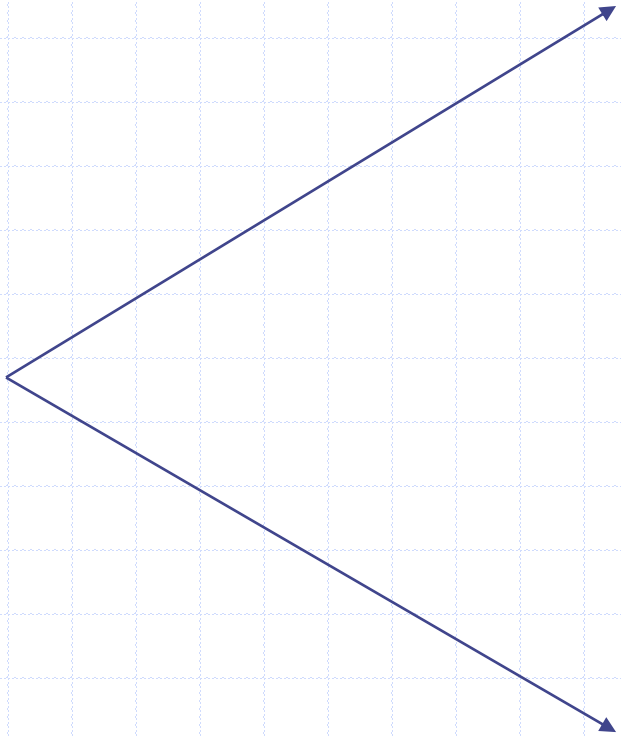


Ако је beq релативан скок.

0001	0xd041	0064
0002	0x73c2	**0003**
0003	0xc001	0001
0004	0xc800	0002
0005	0xc001	0003
0006	0xc800	0004
0034	0xd041	0064
0035	0x73c2	**0003**
0036	0xc001	0001
0037	0xc800	0002
0038	0xc001	0003
0039	0xc800	0004

Смештање у оперативну меморију

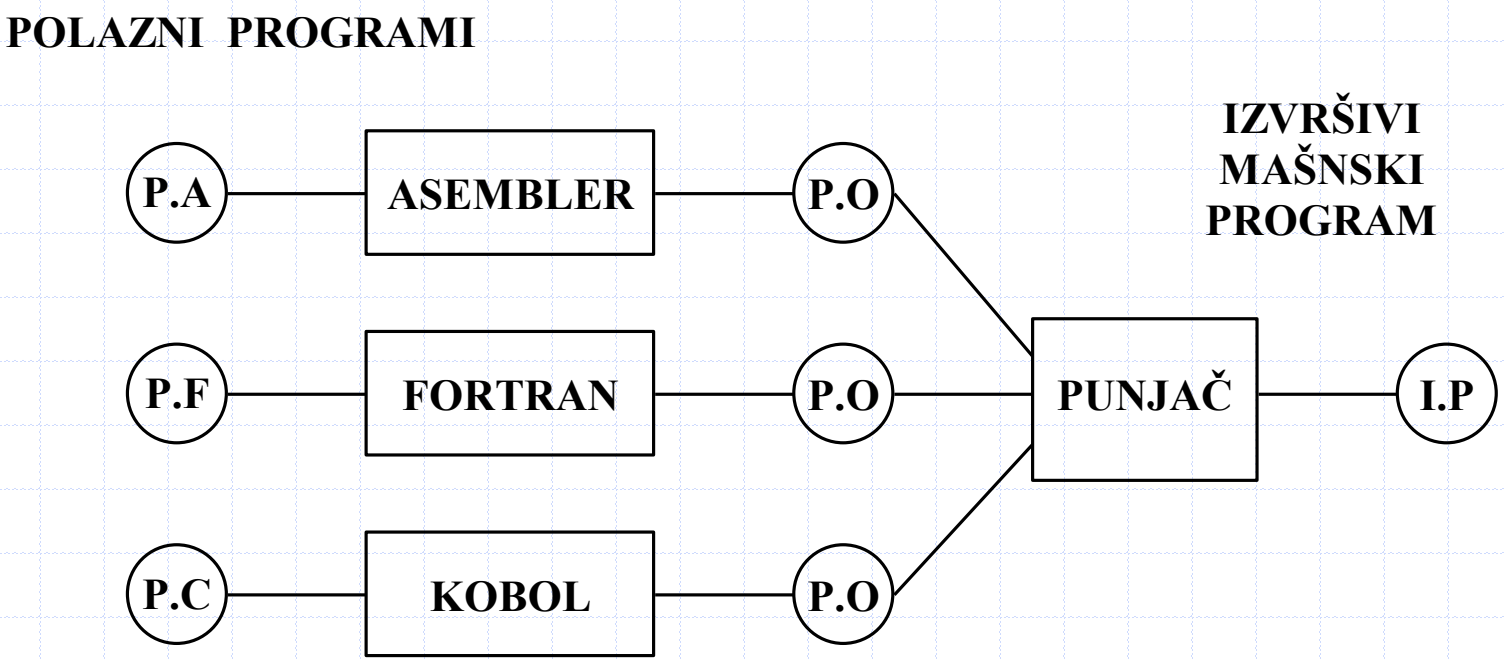
```
Label:
  addc(R1, 3, R0)
  mulc(R0, 4, R0)
  ...
  cmpeqc(R1, 100, R2)
  beq(R2, Label, R30)
  addc(R1, 1, R0)
  mulc(R0, 2, R0)
```



0001	0xc001	0003
0002	0xc800	0004
0003	0xd041	0064
0004	0x73c2	**fffd**
0005	0xc001	0001
0006	0xc800	0002
0034	0xc001	0003
0035	0xc800	0004
0036	0xd041	0064
0037	0x73c2	**fffd**
0038	0xc001	0001
0039	0xc800	0002

Ако је beq релативан скок.

Односи полазних програма, преведених и извршивих машинских програма:



Функције пуњача:

- 1. Додела меморијског простора програмима (алокација);
- 2. Одређивање вредности симболичких референци између релокатибилних програма (повезивање);
- 3. Подешавање свих адресно осетљивих локација на одговарајући додељени простор (релокација);
- 4. Физички пренос машинских инструкција и података у меморију (пуњење).

Типови пуњача

- Преведи и крени.
- Апсолутни.
- BSS.
- Релокатибилни са директним повезивањем.
- Повезивач и пуњач (апсолутне меморијске слике и едитор повезивања).
- Пуњач прекривача.
- Динамички повезивач.

Преведи и крени

- Пуњач у оквиру програма преводиоца: асемблера или компајлера.
- Предност – врло су прости.
- Недостаци: (1) некорисно трошење меморијског простора и процесорског времена (јер асемблер мора бити стално у меморији и програм мора бити преведен пре сваког покретања), и (2) тешко је одржавати већи број различитих сегмената.

Апсолутни пуњач

- Асемблер формира у целини преведени програм.
- То значи да су адресе првих инструкција у програму и процедури дефинисане у време асемблирања.
- Апсолутни пуњач једноставно уноси са спољне меморије програм у дефинисане меморијске локације.

Недостаци апсолутног пуњача

- На одређени начин негира суштину асемблера.
- Немогуће је решити проблем библиотека или програма који се састоји од више независних модула.

Два типа повезивања

- Унутар сегментно или **локално**, које се односи на локације у истом сегменту.
- **Међусегментно**, које се односи на локације у неком другом, спољњем сегменту.
- Стога се **симболи** у процедури класификују или као **локални** или као **глобални**.

ПОВЕЗИВАЊА (BINDING)

- Релокација – дефинисање локалних симбола.
- Повезивање – дефинисање глобалних симбола.

BSS пуњач

- Омогућава пуњење више сегмената програма над само једним заједничким сегментом података.
- Излаз из асемблера је преведени програм и информација о свим другим програмима на које се он позива.
- Наводи се и информација о томе које референце треба мењати (релокациона информација).

Вектор прелаза

foo1:	call **addr1**
foo2:	call **addr2**
bar1:	call **addr3**
…	…
barN:	call **addrX**

Недостаци BSS пуњача:

- Вектор прелаза заузима простор у меморији, а користи се само за повезивање.
- Обрађују сегменте процедура, али не олакшавају приступ сегментима дељивих података.

РЕЛОКАТИБИЛНИ ПУЊАЧ са директним повезивањем

- Универзални релокатибилни пуњач.
- Омогућује пуњење вишеструких сегмената процедуре и вишеструких сегмената података.

Преводилац издаје три типа таблица за сваки сегмент:

- Речник екстерних симбола (ESD) садржи информацију о свим улазним симболима (које користи) и свим излазним симболима (које дефинише).
- Табела текста (TXT) садржи преведену верзију релокатибилног машинског програма.
- Релокациони каталог повезивања (RLD) садржи информацију о свим адресно осетљивим локацијама унутар програма.

Две фазе пуњења:

- Главна улога пуњача у ПРВОЈ ФАЗИ је додела и придруживање меморије сваком сегменту програма и библиотеке и формирање табеле симбола у коју се уносе глобални симболи и апсолутне адресе.
- Главна функција ДРУГЕ ФАЗЕ је пуњење стварног програма и обављање релокационе модификације неке адресне компоненте која тражи ту модификацију.

Повезивач и Пуњач

- Повезивач у ствари обавља функцију доделе, релокације и спајања, а пуњач модула функцију пуњења.
- Постоје две класе повезивача:
- Најпростији тип повезивача развија модул пуњења сличан извршивом коду код апсолутног пуњача.
- Назива се модулом меморијске слике.

Пуњач покривача

- У свим претходним пуњачима претпостављено је да се сви потребни програми и потпрограми пуне у меморију у исто време.
- Шта ако нема довољно меморије?
- Међутим, обично се поједини програми и потпрограми траже у различито време, односно могуће је реализовати структуру ПОКРИВАЧА (overlay).

Едитор повезивања

- Прати релокациону информацију тако да резултујући модул пуњења као целина може на даље да се релоцира и напуни у произвољну зону у меморији.
- Пуњач модула мора да обави додатну доделу и релокацију, као и пуњење, али не решава сложен проблем повезивања.

Динамички повезивач

- Најопштији тип пуњача.
- То је механизам код кога се пуњење и повезивање екстерних референци врши у време извршавања.
- У случају референце на спољну адресу или глобалну променљиву, позива се пуњач, који тек тада пуни сегмент који садржи екстерну референцу.

Повезивање у модерним рачунарима опште намене

- Постоје три начина повезивања:

1) Статичко повезивање (током повезивања)
   - Код сваке спољне функције на крају завршава у извршној датотеци.
   - Позиви функција се обављају директно преко адресе, која се разрешава током повезивања

2) Динамичко повезивање током пуњења
   - Спољне функције се налазе на једном, физички одвојеном месту.
   - Оперативни систем пресликава адресе спољних функција на адресни простор програма који се извршава.
   - Позиви се обављају кроз Procedure Linkage Table (PLT) која се делом генерише током повезивања, али се довршава тек током пуњења (сваки спољни симбол има поље у PLT)

3) Динамичко повезивање током извршавања
   - Нема правих спољних функција.
   - Програм садржи експлицитан код који пресликава адресе неких функција на свој адресни простор.
   - Дакле, показивачи на функције се експлицитно добављају током извршавања.

Повезивање у модерним рачунарима опште намене

- Предности статичког повезивања:
  - Позиви функција су бржи (нема индирекције)
  - Покретање програма је брже - не трошити се време на повезивање
  - Све у једној извршној датотеци - дистрибуција је једноставнија
- Предности динамичког повезивања током пуњења:
  - Мања извршна датотека
  - Нема дуплирања кода уколико више различитих програма користи исту библиотеку (постоји само један примерак библиотеке, а различити процеси/програми је користе – „дељена”)
  - Нове верзије библиотека (унапређене на разне начине) могу бити коришћене у програму без његовог поновног превођења (докле год спрега није промењена)
- Предности динамичког повезивања током извршавања:
  - Време које је потребно за повезивање (и пуњење библиотеке) троши се само ако се током извршавања јави потреба за функционалношћу те библиотеке
  - Програм се може другачије понашати у зависности од тога да ли је нека библиотека присутна или не (нпр. неке функционалности постају доступне кориснику само ако је библиотека присутна, итд.)

Мало детаља о решавању адресно осетљивих локација

```
0x54    beq(R2, Label, R30)          0x73c2 XXXX
...     ...
0x67  Label:
...     ...
0x88    jmp(Label)                   0x5 XXXX 7a1
```

- Три главна податка:
  - Назив симбола
  - Врста инструкције
  - Положај инструкције

Мало детаља о решавању адресно осетљивих локација

- Три главна податка:
  - Назив симбола
  - Врста инструкције
  - Положај инструкције
- Нпр. у ЕЛФ32 формату

```
typedef struct {
Elf32_Addr r_offset;
Elf32_Word r_info;
} Elf32_Rel;
```

  - r_offset – положај инструкције у односу на почетак секције/сегмента
  - r_info – пакована информација о симболу (горњих 8 бита) и врсти инструкције (доњих 8 бита)

Мало детаља о решавању адресно осетљивих локација

```
0x54    beq(R2, Label + 5, R30)          0x73c2 XXXX
...     ...
0x67  Label:
...     ...
0x88    jmp(Label - 2)                   0x5 XXXX 7a1
```

- Три (плус један) главна податка:
  - Назив симбола
  - Врста инструкције
  - Положај инструкције
  - Додатак

Мало детаља о решавању адресно осетљивих локација

- Три главна податка:
  - Назив симбола
  - Врста инструкције
  - Положај инструкције
  - Додатак
- Нпр. у ЕЛФ32 формату

```
typedef struct {
Elf32_Addr r_offset;
Elf32_Word r_info;
Elf32_Sword r_addend;
} Elf32_Rela;
```

  - r_addend – вредност додатка